

Projektdokumentation

F1 Fahrer-Ratespiel

Modul 294 LB Projekt

Name: Donat, Armend

Inhaltsverzeichnis

Inhaltsverzeichnis	2
1. Projektidee	3
2. Anforderungskatalog (User Stories)	3
US 1 – Ratespiel spielen	3
US 2 – Neuen Fahrer erfassen	3
US 3 – Fahrer löschen.....	3
3. Klassendiagramm	3/4
4. Storyboard	4
5. Screen-Mockups	5
6. REST-Schnittstellen	6
7. Testplan	6
8. Testergebnisse	7
9. Installationsanleitung.....	7
10. Hilfestellungen	7

1. Projektidee

F1 Fahrer-Ratespiel ist eine kleine Web-App für Formel-1-Fans. Über einen Start-Button werden zwei zufällige Fahrer gezogen. Vom ersten Fahrer sieht man die Punkte, beim zweiten muss man raten, ob er mehr oder weniger Punkte hat. Bei einem richtigen Tipp hat man gewonnen – dann fährt ein F1-Auto über den Bildschirm und ein Feuerwerk steigt auf. Bei einem falschen Tipp hat man verloren. Über einen Admin-Bereich lassen sich die Fahrer anlegen, bearbeiten und löschen. Es gibt zwei Rollen: einen Spieler, der rät, und einen Admin, der die Daten pflegt.

2. Anforderungskatalog (User Stories)

US 1 – Ratespiel spielen

Als Spieler möchte ich raten, ob der zweite Fahrer mehr oder weniger Punkte als der erste hat, damit ich mein F1-Wissen spielerisch testen kann.

Akzeptanzkriterien:

- Mit einem Start-Button werden zwei zufällige Fahrer gezogen.
- Beim ersten Fahrer sind die Punkte sichtbar, beim zweiten verdeckt.
- Ich kann „mehr“ oder „weniger“ tippen.
- Bei richtigem Tipp wird „Gewonnen“ mit Auto- und Feuerwerk-Animation gezeigt, bei falschem „Verloren“.

US 2 – Neuen Fahrer erfassen

Als Admin möchte ich einen neuen Fahrer anlegen, damit die Liste aktuell bleibt.

Akzeptanzkriterien:

- Es gibt ein Formular mit Name, Team, Punkte und Status.
- Pflichtfeld Name muss ausgefüllt sein, sonst Fehlermeldung.
- Nach dem Speichern erscheint der Fahrer in der Liste.

US 3 – Fahrer löschen

Als Admin möchte ich einen Fahrer löschen, um falsche Einträge zu entfernen.

Akzeptanzkriterien:

- Jeder Fahrer hat einen „Löschen“-Button.
- Vor dem Löschen erscheint eine Bestätigung.
- Nach Bestätigung verschwindet der Fahrer aus der Liste.

3. Klassendiagramm

Mit draw.io erstellen und als Bild einfügen.

Klasse Driver:

Attribut	Typ	Beschreibung
id	String	Eindeutige ID (von json-server)
name	String	Name des Fahrers
team	String	Team des Fahrers
points	int	Karriere-Punkte
status	Status	Enum: aktiv / inaktiv

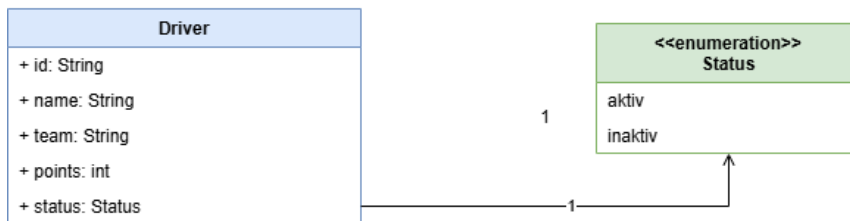
Enum Status: aktiv, inaktiv

Beziehung: Jeder Driver hat genau einen Status (1:1).

Spiellogik (Funktionen, kein eigenes Datenmodell): pickTwoDrivers(drivers) zieht zwei zufällige Fahrer, checkGuess(tipp, a, b) prüft den Tipp.

4. Storyboard

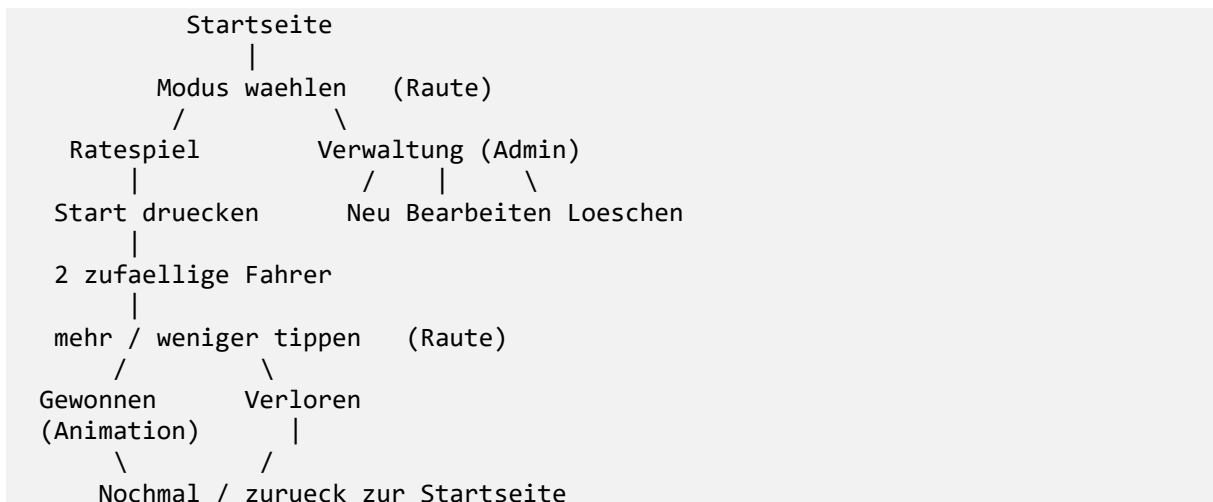
Klassendiagramm – F1 Fahrer-Ratespiel



Spiellogik (reine Funktionen, kein eigenes Datenmodell):

- pickTwoDrivers(drivers) – zieht 2 zufällige Fahrer
- checkGuess(tipp, a, b) – prüft den Tipp (mehr/weniger)
- validateDriver(driver) – prüft Pflichtfelder

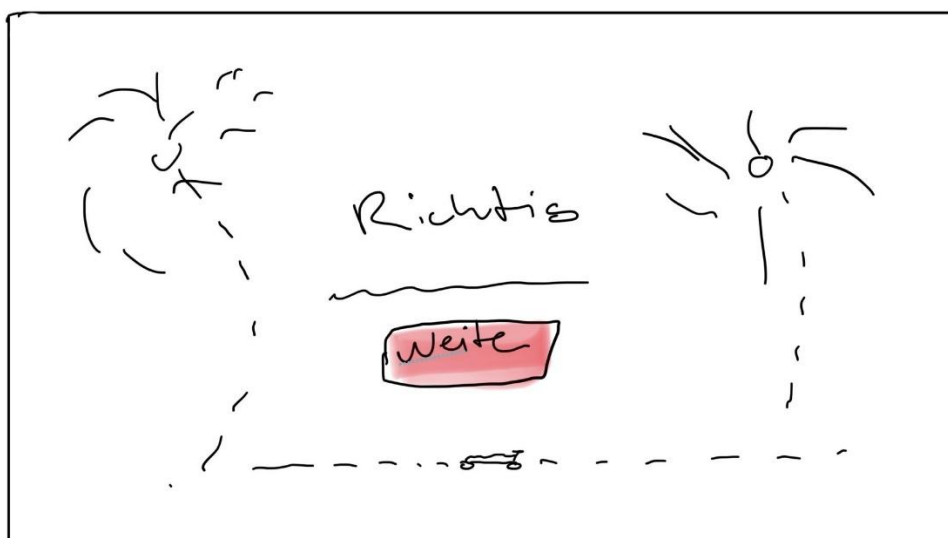
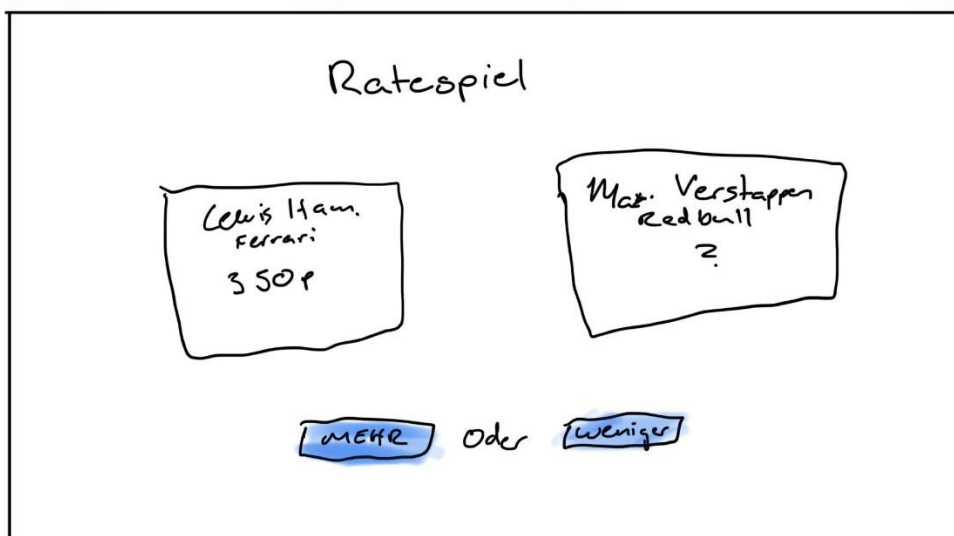
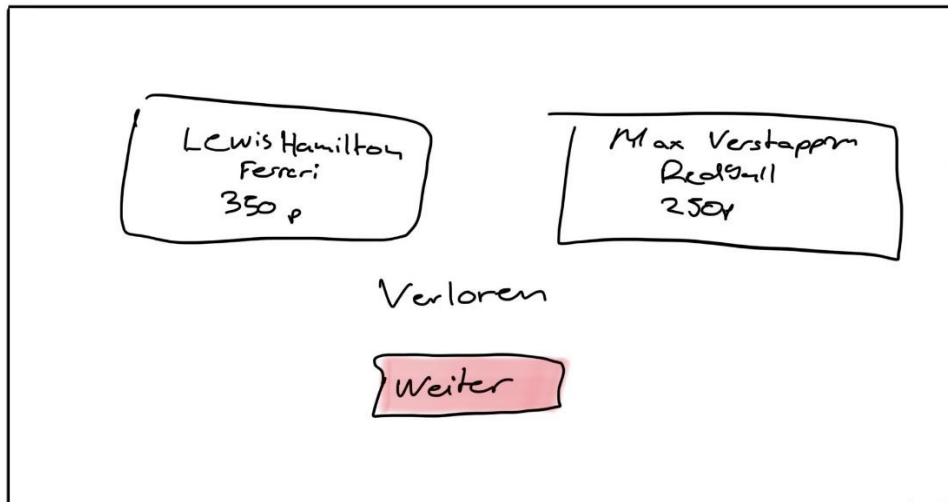
Geplanter Ablauf:



Wichtig:

- Startseite als Einstiegspunkt markiert.
- Entscheidungspunkte (Modus wählen, Tipp) als Rauten.
- Rückweg zur Startseite sichtbar (Nochmal).
- Beide Rollen (Spieler, Admin) abgebildet.

5. Screen-Mockups



6. REST-Schnittstellen

Das Projekt nutzt json-server, um eine REST-API zu simulieren.

Basis-URL (Frontend via VITE_API_URL):

http://localhost:3001

Datentyp Driver:

```
{
  "id": "1",
  "name": "Max Verstappen",
  "team": "Red Bull",
  "points": 2900,
  "status": "aktiv"
}
```

Endpoints:

- GET /drivers = alle Fahrer (200 OK) – wird beim Spielstart und in der Admin-Liste genutzt
- GET /drivers/{id} = einen Fahrer laden (200 OK, 404 falls nicht vorhanden)
- POST /drivers = neuen Fahrer anlegen (201 Created, 400 bei ungültigem Body)
- PUT /drivers/{id} = Fahrer aktualisieren (200 OK, 404 falls nicht vorhanden)
- DELETE /drivers/{id} = Fahrer löschen (200 OK, 404 falls nicht vorhanden)

7. Testplan

T1: Spiel starten

- Vorbedingung: mind. zwei Fahrer mit unterschiedlichen Punkten existieren
- Schritte: Auf Vergleich gehen, Start drücken
- Erwartet: Zwei Fahrer erscheinen, beim ersten sind Punkte sichtbar, beim zweiten verdeckt

T2: Richtig raten

- Vorbedingung: Spiel läuft, zweiter Fahrer hat mehr Punkte
- Schritte: Auf „mehr“ tippen
- Erwartet: „Gewonnen“ wird angezeigt, Auto fährt durch und Feuerwerk erscheint

T3: Falsch raten

- Vorbedingung: Spiel läuft, zweiter Fahrer hat weniger Punkte
- Schritte: Auf „mehr“ tippen
- Erwartet: „Verloren“ wird angezeigt

T4: Neuen Fahrer erfassen mit Validierung

- Vorbedingung: Admin-Seite, Formular offen
- Schritte: Name leer lassen und speichern; danach Name ausfüllen und speichern
- Erwartet: Erst Fehlermeldung (nicht gespeichert), dann erscheint der Fahrer in der Liste

T5: Fahrer löschen

- Vorbedingung: mind. ein Fahrer existiert
- Schritte: Löschen klicken, bestätigen
- Erwartet: Fahrer verschwindet aus der Liste

8. Testergebnisse

Nach Durchführung ausfüllen.

ID	Testfall	Status	Bemerkung
T1	Spiel starten	Erfolgreich	Beide Fahrer erscheinen und einer hat Fragezeichen als punkte.
T2	Richtig raten	Erfolgreich	Gewonnen und Feuerwerke funktionieren. Auto fuhr im ersten Test rückwärts aber jetzt stimmt.
T3	Falsch raten	Erfolgreich	Verloren wurde angezeigt.
T4	Erfassen + Validierung	Erfolgreich	Funktioniert problemlos.
T5	Fahrer löschen	Erfolgreich	Funktioniert problemlos.

9. Installationsanleitung

Voraussetzung: Node.js 18+

1. ZIP in Ordner Front-End/ wechseln.
2. npm install
3. API starten: npm run server (json-server auf Port 3001)
4. Frontend starten: npm run dev
5. Browser öffnen: <http://localhost:5173>
6. Tests ausführen: npm test

10. Hilfestellungen

KI, Quellen und Personen notieren.

- KI-Tools: Claude für schöneres UI und Korrektur lesen + Fehlersuche.
- Internet-Quellen: React-Doku, json-server-Doku, F1-Daten 2026
- Personen: Donat, Armend